AWS Data Engineer: Interview Preparation

Role fit

Verified: Python, SQL, S3, Glue/PySpark, Lambda, Step Functions, SQS, DynamoDB, Redshift, monitoring and REST APIs.

Gaps to disclose

No verified hands-on Databricks, Apache Airflow, Kafka, NiFi, Snowflake, MongoDB, feature stores, or production ML deployment.

ATS estimate

Approximately 75/100 for the aggregate AWS data-engineering skills; posting-specific fit varies widely.

60-Second and 2-Minute Introduction

60 seconds

I build Python-based data and serverless AWS workflows, focusing on reliable ETL, incremental loads, SQL and REST integration. Recent finance projects used Glue/PySpark, S3 and Step Functions; another modernized document processes with Lambda and SQS.

Two minutes

Expand with an example of schema mapping, quality checks, job bookmarks, monitoring and alerts; explain your own role, tradeoffs and measurable project outcomes. End with the AWS platform and pipeline reliability work you want to own.

Project Deep Dive: Finance ETL

Architecture

Source finance data -> S3 landing -> Glue/PySpark mapping and validation -> downstream storage -> Step Functions orchestration -> CloudWatch and alerts.

Expected probing

Describe approximately 30GB+ daily volume, partition approach, job bookmarks, idempotency and rejected records. Explain how retries avoid duplicates.

Ownership and outcome

Discuss your own implementation and the reported 40% reduction in downtime, without implying unsupported scale or precision.

Project Deep Dive: Document Automation

Architecture

Oracle ORDS/REST -> Lambda -> SQS for decoupling -> enrichment with DynamoDB -> PDF/CSV outputs in S3.

Expected probing

Queue retries and dead-letter design, duplicate document prevention, conditional templates, reprint audit trail, bucket permissions.

Outcome

Describe the documented 80% manual-effort reduction, what was automated, and how exceptions are handled.

SQL, Python and PySpark Q&A

SQL

Explain joins, GROUP BY vs window functions, indexes, EXPLAIN plans, deduplication using ROW_NUMBER, and safe incremental merge patterns.

Python

Discuss generators, iterators, context managers, exceptions, logging, typing, memory-safe file streams, pytest fixtures.

PySpark

Explain lazy evaluation, shuffle, partitioning, skew, broadcast joins, caching, schema enforcement and null handling.

AWS Services and Pipeline Design

S3 / IAM

Explain zone prefixes, encryption, least-privilege IAM, versioning, object naming, partitioning and lifecycle policies.

Glue / orchestration

Describe Glue jobs/bookmarks, Step Functions retries, failure branches and batch reconciliation.

Lambda / SQS

Describe at-least-once message delivery, visibility timeout, dead-letter queues, idempotent processors and concurrency limits.

Troubleshooting and Production Scenarios

Broken nightly ETL

Check CloudWatch errors, source freshness, job-run metadata, schema drift, retry history, downstream counts and replay checkpoint.

Slow query / Spark job

Inspect query plan, distribution, shuffle/skew, partition pruning and small-file behavior; change one bottleneck at a time.

Incorrect duplicates

Trace upstream event keys and replay windows; add deduplication, business-key constraints and reconciled checkpoints.

System Design: AWS Finance Data Platform

Ingestion

Land files/API responses in S3 with timestamped manifests; validate contracts, distinguish full vs incremental sync and quarantine failures.

Transform

Use Glue/PySpark for data cleaning and conforming; partition curated data and route controlled outputs to Redshift.

Operations

Use Step Functions for orchestration, CloudWatch for alerts, IAM/KMS for security, quality metrics and SLA reporting.

Coding Frameworks & Practice

Python exercise

Read NDJSON incrementally, validate required finance fields, deduplicate on composite keys, and output rejected-record reasons.

SQL exercise

Use ROW_NUMBER() OVER (PARTITION BY invoice_id ORDER BY updated_at DESC) to select the latest invoice version.

PySpark exercise

Create typed schema, filter bad records, join dimension tables, deduplicate by transaction ID, write partitioned Parquet.

Behavioral STAR Practice

Example 1

Situation: legacy manual document process. Task: automate and standardize it. Action: Lambda/SQS/S3 processing and templating. Result: documented manual reduction.

Example 2

Situation: finance-data pipeline instability. Task: improve recovery and visibility. Action: validation, CloudWatch alerting and retries. Result: reported downtime reduction.

Follow-up

Be precise about personally owned components, collaboration, tradeoffs, production errors, and lessons learned.

Gap Handling & POC Learning Plan

Databricks / Delta

Build a sample bronze-silver-gold lakehouse with Delta MERGE and quality checks; label as personal learning, not prior work.

Airflow / Kafka / NiFi

Create a small Airflow DAG and Kafka event consumer; compare their semantics to Step Functions and SQS.

MLOps / feature stores

Study point-in-time correctness, offline/online features and CI/CD deployment concepts; do not imply production ownership.

Final Revision Checklist

Project stories

Rehearse each project in 2 minutes: objective, architecture, your role, tradeoff, error-handling, measurable result.

Technical drill

Prepare one SQL query, one PySpark transform and one Python file-processing implementation without AI assistance.

Eligibility

Verify onsite locations, work authorization and experience threshold for each specific posting before applying.